Université de Haute-Alsace
École Nationale Supérieure d'Ingénieurs
Sud Alsace

Informatique et Réseaux - 2A

Artificial Intelligence and Reinforcement Learning

TD1 : Classical Search

Exercise 1

— Modeling a Search Problem

A Pokémon trainer wants to reach the nearest Pokémon Center (Clinic) before continuing their adventure. The map of the area is shown in Figure 1. The trainer may move one cell at a time in one of the four cardinal directions (up, down, left, or right). Trees are obstacles and cannot be crossed, whereas grass cells are traversable.



Figure 1: Pokémon map used throughout this exercise.

1. Identify the following components of this search problem:
   (a) The agent.
   (b) The environment.
   (c) The initial state.
   (d) The goal state.
   (e) The set of possible actions.

2. Describe the transition model of the environment. In particular, explain what happens when the trainer attempts to move:
   (a) onto a grass cell;
   (b) onto a tree;
   (c) outside the map.

3. Suppose each valid movement has a cost of 1.
   (a) What is the cost of remaining in the same position?
   (b) How is the cost of a complete path computed?

4. Is the trainer's position alone sufficient to represent the state in this problem? Justify your answer.

5. Consider now a modified version of the game where the trainer must first collect a Gym Badge before entering the Pokémon Center.

   Would the trainer's position still be sufficient to describe the state? If not, propose a suitable state representation.

Exercise 2

— Exploring the State Space

We consider the same Pokémon map shown in Figure 1. The trainer may move one cell at a time in one of the four cardinal directions. Trees remain impassable.

For this exercise, rows are numbered from top to bottom and columns from left to right, starting from 1. A trainer position is represented by a pair

[(row,column).]

1. Determine the coordinates of:

   (a) the initial state;

   (b) the goal state.

2. From the initial state, list all valid actions available to the trainer. For each valid action, give the resulting state.

3. Starting from the initial state, list all distinct states that can be reached after exactly:

   (a) one valid action;

   (b) two valid actions.

   A state reached through several different action sequences should only be listed once.

4. Consider the following action sequence:

   [right, right, down, down, right.]

   (a) Apply the actions one after another from the initial state.

   (b) Give the trainer's state after each action.

   (c) Indicate whether every action is valid.

   (d) If an action is invalid, explain why.

5. Can the trainer return to a state that has already been visited? Give an example.

   Why could it be useful for a search algorithm to remember previously visited states?

6. Suppose the trainer follows a sequence containing n valid movements.

   (a) What is the path cost?

   (b) Can two different paths leading to the same state have different costs? Explain.

Exercise 3
— Breadth-First and Depth-First Exploration

Professor Oak wants to equip the trainer with an automatic navigation system capable of finding the Pokémon Center shown in Figure 1.

The complete state space is not constructed in advance. Whenever a state is explored, the navigation system generates the valid neighboring states by considering the actions in the following order:

$$\text{up} \longrightarrow \text{right} \longrightarrow \text{down} \longrightarrow \text{left}.$$

A state that has already been explored, or that is already present in the frontier, is not added again.

The goal test is performed when a state is removed from the frontier.

1. Apply **Breadth-First Search (BFS)** from the initial state until the Pokémon Center is reached.

   Recall that BFS stores the frontier in a **queue**: the first state added is the first state removed.

   During the execution of the algorithm, keep track of the frontier and the explored states. Then provide:

   (a) the order in which the states are removed from the frontier;
   (b) the path returned by BFS as a sequence of states;
   (c) the corresponding sequence of actions;
   (d) the cost of the returned path;
   (e) the number of states removed from the frontier, including the goal state.

2. Draw the path returned by BFS on a copy of Figure 1.

3. Apply **Depth-First Search (DFS)** to the same problem.

   Recall that DFS stores the frontier in a **stack**: the last state added is the first state removed.

   When several successors are generated, they must be inserted into the stack so that the following priority is respected when states are removed:

   $$\text{up}, \text{right}, \text{down}, \text{left}.$$

   During the execution of the algorithm, keep track of the frontier and the explored states. Then provide:

   (a) the order in which the states are removed from the frontier;
   (b) the path returned by DFS as a sequence of states;
   (c) the corresponding sequence of actions;
   (d) the cost of the returned path;
   (e) the number of states removed from the frontier, including the goal state.

4. Draw the path returned by DFS on a copy of Figure 1.

5. Compare the results obtained using BFS and DFS.

   (a) Which algorithm returns the shortest path in this example?
   (b) Which algorithm removes fewer states from the frontier?

(c) Is DFS guaranteed to return a shortest path?

(d) Is BFS guaranteed to return a shortest path in this problem?

Exercise 4
— Programming Breadth-First and Depth-First Search

In this exercise, you will implement the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms introduced during the lecture.

A Python project is provided with this TD [1]. It contains a small Pokémon environment where a trainer must automatically navigate from an initial position to a Pokémon Center.

The project already provides:

- random generation of valid Pokémon maps;
- visualization of the map;
- animation of a trainer following a valid path;
- generation of valid successor states;
- path validation utilities.

Your task is to implement the search algorithms only.

Project structure

- `pokemon_game.py`: environment and visualization utilities.
- `search.py`: implement the search algorithms.
- `main.py`: example program used to test your implementation.

Required Python version

The project was developed using `Python 3.12.3` (or any compatible Python 3.12 version).

Installing the required packages

A `requirements.txt` file is provided with the project. Install the required Python packages by executing:

```
python3 -m pip install -r requirements.txt
```

The project depends on the following Python packages:

- NumPy
- Matplotlib
- Pillow

[1] https://hadifawaz1999.github.io/Artificial-Intelligence-Reinforcement-Learning--2A-Info/TD/TD_01_classical_search/exo4.zip

- CairoSVG

Ubuntu users

If you are using Ubuntu, you may also need to install a few system packages before running the project:

```
sudo apt update
```

```
sudo apt install python3-tk libcairo2 libffi-dev
```

These packages enable the graphical interface used to display and animate the Pokémon environment.

Your tasks

Implement the following two functions contained in `search.py`:

- `breadth_first_search(...)`
- `depth_first_search(...)`

Each function must return a path represented as a list of states, beginning with the initial state and ending with the goal state.

Once a valid path is returned, the provided visualization tools will automatically animate the trainer walking along the path.

Expected behavior

After implementing the two algorithms:

- Breadth-First Search should always return a shortest path.
- Depth-First Search should return a valid path, although it may not be the shortest one.